STAT3006 Worksheet 05

1. Random Forest: Splitting Bias and Feature Subsampling

Consider a random forest regression estimator

$$f_{RF}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} f^{(b)}(\mathbf{x})$$

where each tree is grown fully and uses feature subsampling of size q at each split.

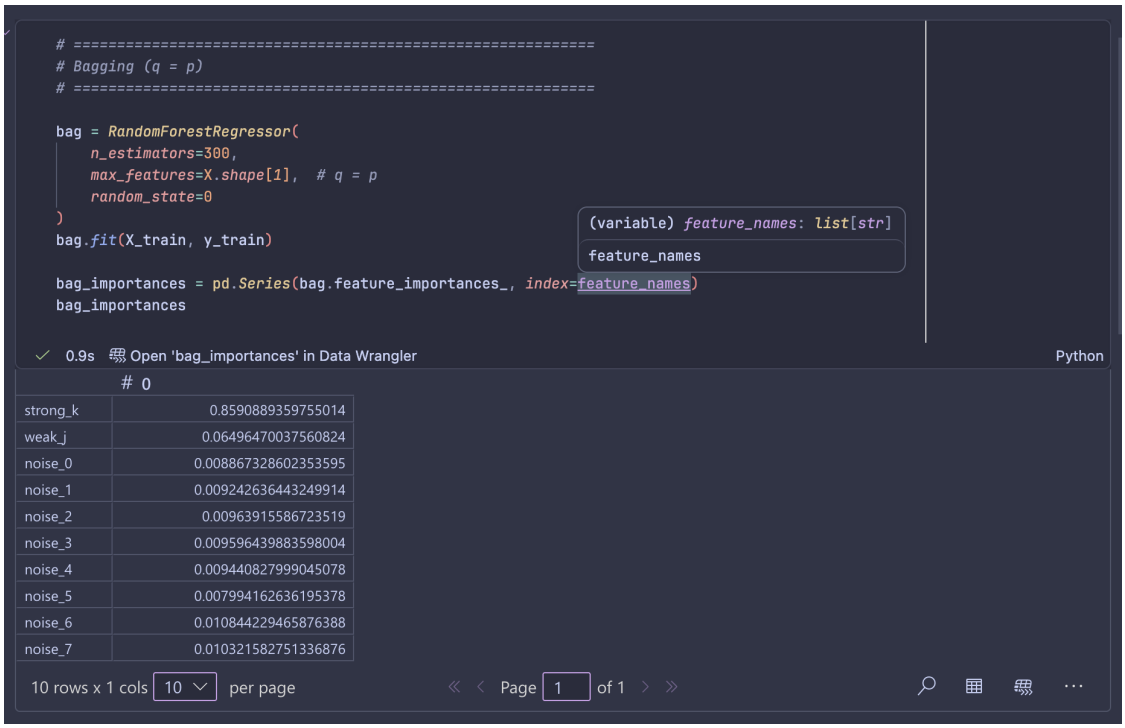
(a) Consider a feature $\mathbf{x}_j$ that is weakly predictive but correlated with a strong feature $\mathbf{x}_k$. Explain why bagging tends to underuse $\mathbf{x}_j$

Answer: The bagging algorithm works by generating B subsamples of the training data by picking random datapoints with replacement. It then fits B trees for each subsample.

Since $\mathbf{x}_k$ is a strong feature, we can expect most, if not all B trees to choose $\mathbf{x}_k$ at the first split, in which the CART algorithm might reach its stopping condition before it ends up choosing the weakly predictive feature $\mathbf{x}_j$. In other words, it underuses $\mathbf{x}_j$

(b) Run the attached Jupyter Notebook titled "WS.ipynb" which simulates the above situation and confirm your observations

Answer: Consider the following code snippet and output

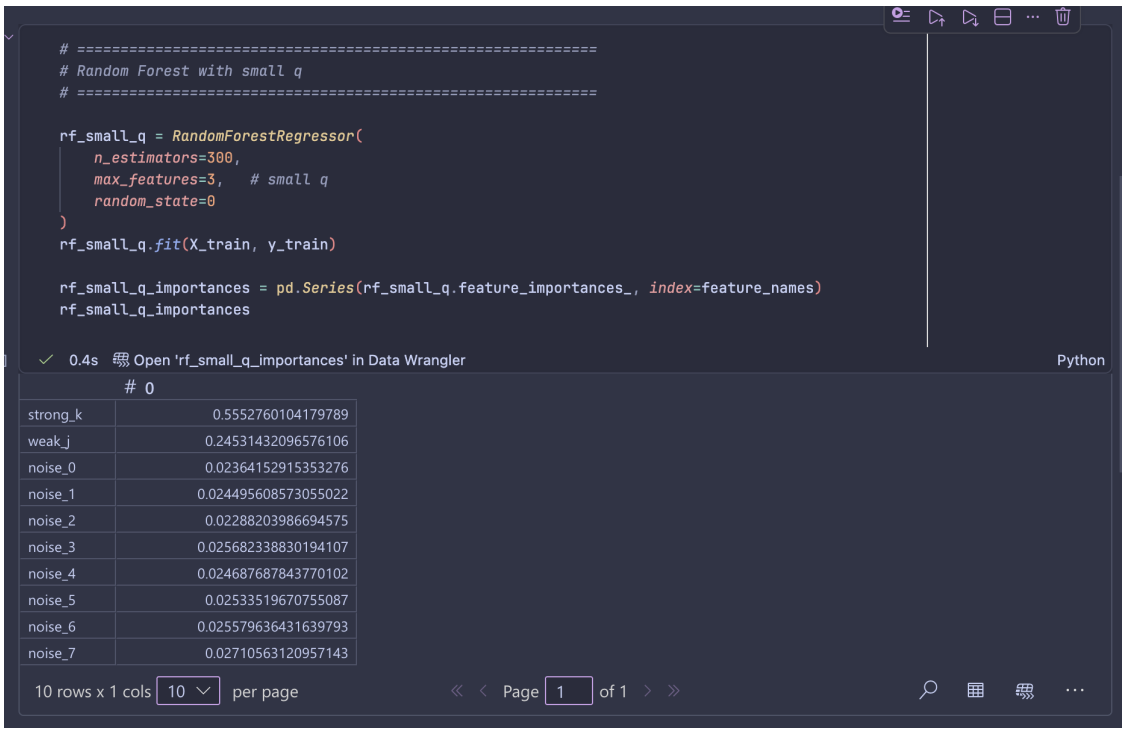


Here we can see that a most of the feature importance is placed in the strong predictor $\mathbf{x}_k$ in the bagging algorithm, confirming our observations from part (a).

(c) Explain how feature subsampling in random forests mitigates the issue in part (a)

Answer: In the Random Forest algorithm, it works similarly to bagging where B subsamples of the training data are generated at random with replacement. However for each tree, we also pick a subset of the original features of length $q, q \leq p$ that we use to train the tree.

This mitigates the issue found in part (a) as in some trees, the feature subset may exclude the strong feature but include the weak predictor. We confirm our observation by running the below code snippet:



```
# ============================================================
# Random Forest with small q
# ============================================================

rf_small_q = RandomForestRegressor(
    n_estimators=300,
    max_features=3,   # small q
    random_state=0
)
rf_small_q.fit(X_train, y_train)

rf_small_q_importances = pd.Series(rf_small_q.feature_importances_, index=feature_names)
rf_small_q_importances
```

✓ 0.4s Open 'rf_small_q_importances' in Data Wrangler — Python

	# 0
strong_k	0.5552760104179789
weak_j	0.24531432096576106
noise_0	0.02364152915353276
noise_1	0.024495608573055022
noise_2	0.02288203986694575
noise_3	0.025682338830194107
noise_4	0.024687687843770102
noise_5	0.02533519670755087
noise_6	0.025579636431639793
noise_7	0.02710563120957143

10 rows x 1 cols 10 per page « ‹ Page 1 of 1 › »

(d) Suppose q is extremely small (e.g. $q = 1$). Discuss the bias-variance trade-off in this extreme regime.

Answer: Since every tree has only one feature, there is no greedy behaviour where the CART algorithm picks the best feature to determine the best split, it just chooses the single feature that it is given. Since that single feature is chosen at random, the splits are essentially random and therefore the trees will have low correlation. Recall the variance of an ensemble,

$$\frac{1-\rho}{B}\sigma^2 + \rho\sigma^2$$

Since $\rho \approx 0$, as $B \to \infty$ we decrease the overall variance of the ensemble down to zero. On the other hand, since each tree uses its given feature at the split point it will train the tree on just that one feature. In other words, each tree in the ensemble has high bias, and since the bias of the ensemble is just the average of the bias of it's trees, since all trees have high bias then the ensemble will also have high bias.

(e) Run the attached Jupyter Notebook titled "q.ipynb", which studies the effect of q on bias,

variance, and predictive performance under two different data-generating regimes

- Regime A: (Sparse Strong Signal)

$$f_A(x) = \sin(x_0) + 0.5x_1$$

  Where only two features carry signal and the remaining $p - 2$ features are pure noise

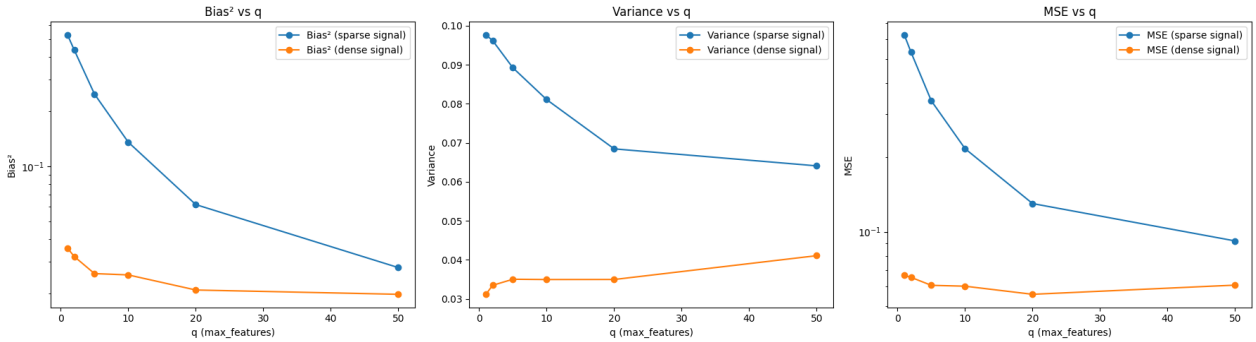
- Regime B: (Dense Weak Signal)

$$f_B(x) = \frac{1}{10}\sum_{j=0}^{9} \sin(x_j)$$

  Where ten features contribute weakly and the remaining $p - 10$ features are noise.

Explain your observations

Answer: Consider the following results:



Bias: For smaller values of q, the random forest algorithm can only choose from the subset of p features to split, which leads to high bias for each individual trees. This leads to high bias for the whole ensemble. As q grows, each tree becomes more flexible and therefore bias decreases.

Variance: For smaller q, the sparse signal ensemble results in high variance as for a significant number of trees, it will most likely pick the noisy features $(p - 2)$. As q grows, there is a higher chance that the two strong features are picked in which the CART algorithm will pick these to split on, therefore reducing variance as the tree captures the behaviour of the data generating distribution.

On the other hand, we can see that the dense singal starts at low variance for small q. As explained in the previous question, this results in low variance as $\rho \approx 0$, therefore the variance of the ensemble can be determined via

$$\frac{1}{B}\sigma^2$$

However as q grows, ρ increases and leads to higher variance. Another explanation is that since the 10 features used contribute weakly, adding other noisy features to the training algorithm will make the tree learn the noise as opposed to the true data-generating distribution, leading to higher variance.

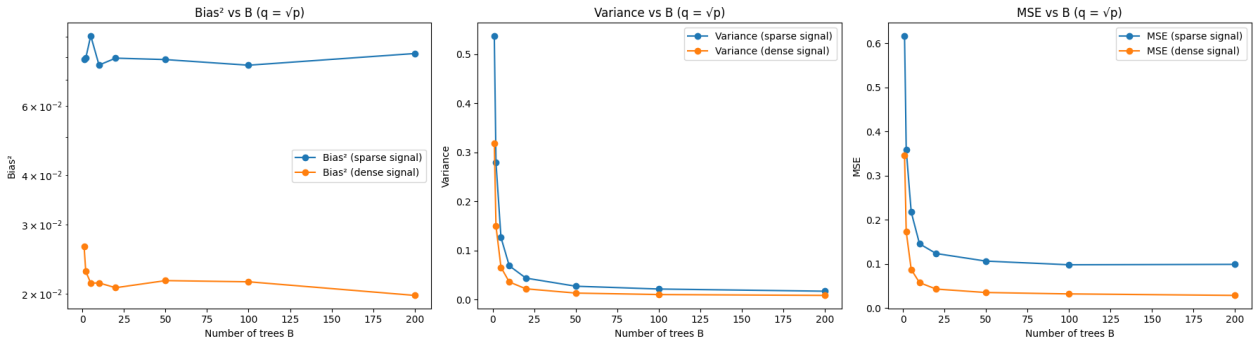
MSE: MSE is formulated by

$$\text{MSE} = \text{Variance} + \text{Bias}^2$$

Using this equation, we can trivially reason about why the MSE behaves as seen above by looking at how the variance and bias squared behaves for each data generating distribution.

(f) Now explore the role of te number of trees B in terms of bias, variance and prediction performance. Run the Jupyter Notebook titled "B.ipynb", which fits random forests with varying B to both regression functions above. What do you observe? Explain your findings.

Answer: Consider the following outputs:



Bias: We can see here that for both data-generating distributions, bias stays about the same for all values of B, this is because bias is heavily dependent on the size of the feature subset q which in this experiment is a constant value.

Variance: Again recall the equation for the variance of an ensemble

$$\frac{1-\rho}{B}\sigma^2 + \rho\sigma^2$$

As B increases, the first term approaches zero in which the variance of the ensemble mostly dependends on the second term.

MSE: Again we can trivially reason about the behaviour of the MSE by looking at the variance and bias.